



UNIVERSITY OF ASIA PACIFIC

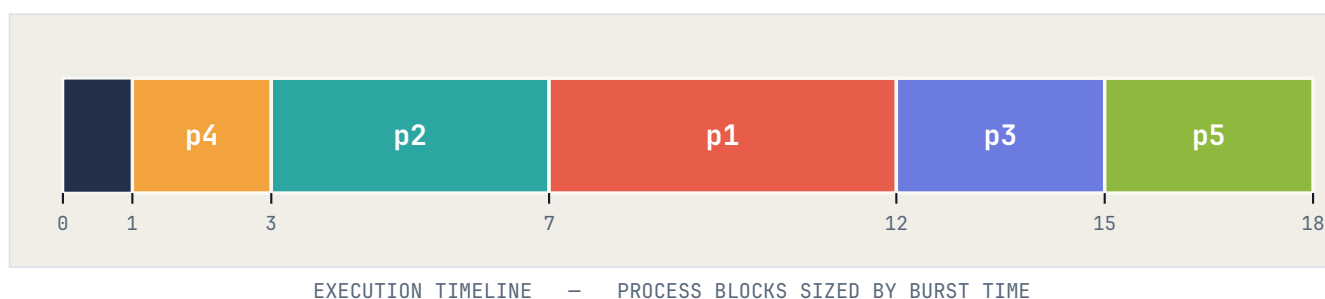
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

OPERATING SYSTEM LAB · REPORT 01

August 13, 2026

FCFS SCHEDULING

First-Come, First-Served CPU scheduling



COURSE CODE
CSE 402

LAB REPORT
01

COURSE NAME
Operating System Lab

EXPERIMENT
First Come First Serve Scheduling

CODE REPOSITORY
github.com/ashadsetu/CSE-402

SUBMITTED BY

SUBMITTED TO

MD Asadullah

Reg. No: 23101126 Section: C1
Semester: 4th Year 1st Semester

Atia Rahman Orthi

Lecturer, Department of CSE
University of Asia Pacific

PROBLEM STATEMENT

Implement FCFS CPU scheduling for the 5 processes below and find CT, TAT, WT, their averages, and the execution sequence.

PID	AT	BT
 p4	1	2
 p2	2	4
 p1	3	5
 p3	4	3
 p5	5	3

CODE

```
pid = ["p4", "p2", "p1", "p3", "p5"]
AT = [1, 2, 3, 4, 5]
BT = [2, 4, 5, 3, 3]

CT = []
TAT = []
WT = []

ct = 0

for i in range(5):
    if ct < AT[i]:
        ct = AT[i]
    ct = ct + BT[i]
    CT.append(ct)

for i in range(5):
    tat = CT[i] - AT[i]
    TAT.append(tat)

    wt = tat - BT[i]
    WT.append(wt)

print(f"{'PID':<5}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")

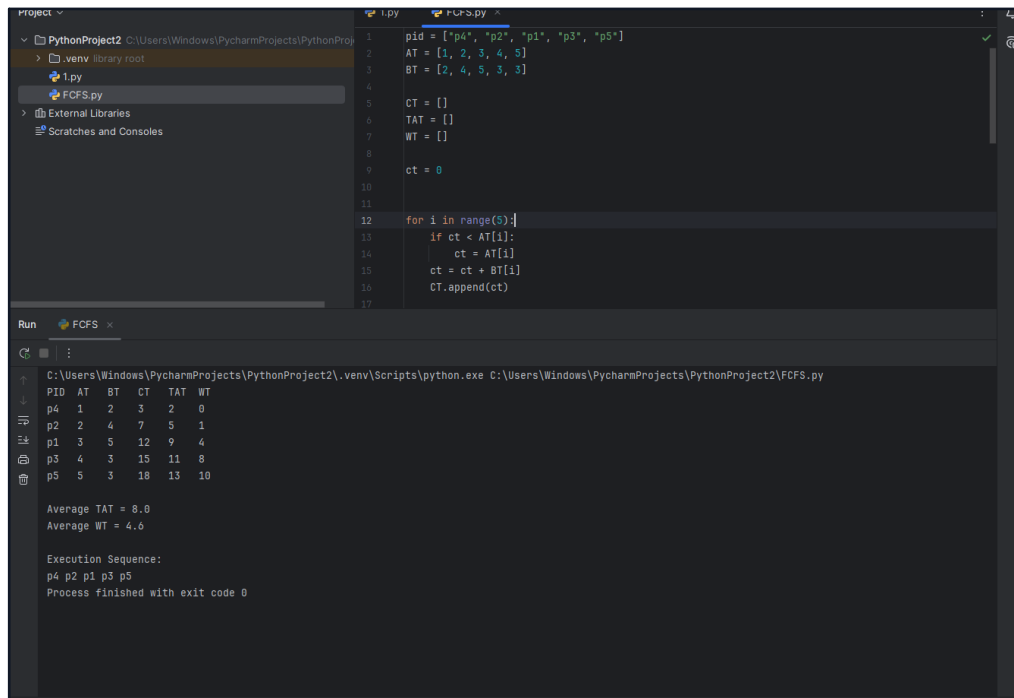
for i in range(5):
    print(f"{pid[i]:<5}{AT[i]:<5}{BT[i]:<5}{CT[i]:<5}{TAT[i]:<5}{WT[i]:<5}")

avg_tat = sum(TAT) / 5
avg_wt = sum(WT) / 5

print("\nAverage TAT =", avg_tat)
print("Average WT =", avg_wt)

print("\nExecution Sequence:")
for i in range(5):
    print(pid[i], end=" ")
```

OUTPUT



The screenshot shows the PyCharm IDE with a project named 'PythonProject2'. The code editor displays the 'FCFS.py' file with the following Python code:

```
1 pid = ["p4", "p2", "p1", "p3", "p5"]
2 AT = [1, 2, 3, 4, 5]
3 BT = [2, 4, 5, 3, 3]
4
5 CT = []
6 TAT = []
7 WT = []
8
9 ct = 0
10
11
12 for i in range(5):
13     if ct < AT[i]:
14         ct = AT[i]
15     ct = ct + BT[i]
16     CT.append(ct)
17
```

The Run console shows the output of the program:

```
C:\Users\Windows\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\Windows\PycharmProjects\PythonProject2\FCFS.py
PID AT BT CT TAT WT
p4 1 2 3 2 0
p2 2 4 7 5 1
p1 3 5 12 9 4
p3 4 3 15 11 8
p5 5 3 18 13 10

Average TAT = 8.0
Average WT = 4.6

Execution Sequence:
p4 p2 p1 p3 p5
Process finished with exit code 0
```

PYCHARM CONSOLE OUTPUT FOR THE FCFS PROGRAM

DISCUSSION

At first the algorithm seemed simple, but a few small things came up while coding it.

- The processes were already listed in arrival order (p4, p2, p1, p3, p5 with AT = 1, 2, 3, 4, 5), so we didn't need a sorting step, the array order already matched arrival order.
- We kept the line 'if ct < AT[i]: ct = AT[i]' even though the CPU never actually sits idle here. Taking it out would've only worked for this one input, and the assignment needed it to work for any AT/BT combination.
- Our first print used plain tabs, and the TAT column kept printing shifted out to the right of the other values whenever a number had a different digit count, throwing the whole table out of line. Switching to f-string field-width formatting (f"{pid[i]:<5}") fixed the alignment.
- We briefly used TAT – AT instead of TAT – BT for waiting time, then caught the mistake and checked it against our hand trace (CT: 3, 7, 12, 15, 18, TAT: 2, 5, 9, 11, 13, WT: 0, 1, 4, 8, 10).
- p5 ends up waiting 10 units even though its own burst time is only 3, purely because four other processes got serviced first. It's a clear example of the convoy effect.

After fixing these small issues, the program ran fine in PyCharm and gave Average TAT = 8.0 and Average WT = 4.6, matching what we calculated by hand.

CONCLUSION

Overall the small mistakes were worth catching. Fixing the idle check, the alignment, and the TAT/WT mix-up is what got us matching numbers. Watching p5 wait 10 units for a burst time of just 3 made the convoy effect click more than just reading about it, and the program's output lining up with our hand trace confirmed the logic was actually correct, not just the formatting.